Episodio 9 de 14: ¿Es correcto? Detectores y la cota wait-free

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 8, «Latencia y memoria retenida».
- **Episodio siguiente:** episodio 10, «Sin Docker: el error escondido y la caché que se llena».

Lo esencial de este episodio:

- ASan y TSan no encontraron errores en Autores. La versión ingenua los dispara enseguida, lo que demuestra que los detectores funcionaban.
- Romper a propósito las reglas del paper produce errores: con C = P aparecieron 42 348.
- La cota se cumple con margen: en 8 061 142 recargas forzadas hubo como mucho 8 intentos, con una cota de 32. Aun así, encontrar 0 errores no demuestra que no existan.

6.5 Corrección: funciona sin romperse

Ser rápido no vale de nada si el programa rompe la memoria. Por eso se usaron detectores automáticos. **AddressSanitizer (ASan)** vigila los accesos a memoria ya liberada. **ThreadSanitizer (TSan)** vigila los accesos simultáneos sin protección.

Prueba	Versión ingenua	Autores
ASan, con 8, 12 y 16 hilos	`double-free`: el detector la para enseguida	0 errores
TSan, con 8 y 16 hilos	carreras de datos y `heap-use-after-free`	0 avisos
Boceto didáctico (`codigo/boceto/`)	227 031 errores	0 errores, 0 objetos sin liberar

¿Y los rivales? Al principio estas pruebas solo se pasaron a Autores. Después se pasaron a todos con `herramientas/sanitizers_rivales.sh`, y a ASan se le sumó **UBSan**, que vigila operaciones que C++ no define, como desbordar un entero.

Prueba	Algoritmos	Ejecuciones	Avisos
ASan + UBSan sobre `bench.cpp`, con 8 y 16 hilos	Autores, Libstdc++, Williams, Mutex	12	0
TSan sobre `bench.cpp`, con 8 hilos	Autores, Libstdc++, Williams, Mutex	4	0
ASan + UBSan sobre el `bench_stack` oficial, con 8 y 16 hilos, en Docker	Autores, Libstdc++, Williams, Folly, Anderson	10	0

Dicho sencillo: ninguno de los seis rompió la memoria en estas pruebas. Esto es importante por una razón: si un rival hubiera ganado usando memoria ya liberada, su victoria no valdría.

Tres avisos para no leer de más:

- **«0 avisos» solo vale si el detector estaba puesto.** Se comprobó que los tres binarios llevan la instrumentación, y la versión ingenua demuestra que el detector sí salta cuando hay un error.
- **Folly y Anderson no pasaron TSan.** `libfolly.a` no está instrumentada y daría avisos falsos.
- **Son 4 segundos por prueba y un solo escenario de contención fuerte.** Un detector solo ve lo que ocurre mientras mira. Encontrar 0 errores no demuestra que no existan.

El boceto permite además **romper las reglas a propósito** para ver si de verdad hacían falta:

Variante	Resultado	Lección
Quitar la salida «otro ya recargó»	0 errores, pero **45 intentos**, con P = 8	Sigue siendo correcto, pero deja de tener cota: ya no es wait-free
Poner C = P, violando R-1	**42 348 errores**, 4 280 objetos sin liberar	La regla C ≥ 2P no es un adorno

La cota de intentos, en el código real. La gran promesa del paper es: «la recarga reintenta como mucho P veces». Para comprobarlo, `herramientas/correr_cota_cas.sh` añade un contador a una copia del código de los autores.

Con los valores reales (P = 65 536) casi nunca hay recargas, porque el taco de 524 287 vales dura muchísimo. Hubo como máximo 4 intentos, y solo con 0 % de `store`. Para forzar la situación se bajó el taco: P = 32 y C = 64.

Variante	Recargas (hasta)	Máximo de intentos	Cota	Errores
Real: P = 65 536, C = 524 287	1 158	4	65 536	0
Forzada: P = 32, C = 64	8 061 142	**8**	32	0
Límite: P = 32, C = 32 (viola R-1)	15 600 117	10	32	0

Millones de recargas y el máximo se queda muy lejos de la cota. Ojo con la tercera fila: C = P no falló aquí porque había menos hilos que P, pero recarga en casi todas las operaciones y rinde aproximadamente la mitad. Estas pruebas no demuestran que el algoritmo sea correcto, porque una prueba nunca demuestra eso. Solo dicen que, buscando el fallo con ganas, no apareció.